

OPTIMISTIC VERSUS PESSIMISTIC CONCURRENCY CONTROL MECHANISMS IN DATABASE MANAGEMENT SYSTEMS

DANIEL A. MENASCÉ and TATUO NAKANISHI

Departamento de Informática, Pontifícia Universidade Católica do Rio de Janeiro, Brasil

(Received 11 August 1980; in revised form 1 June 1981)

Abstract—Concurrency control mechanisms are necessary to preserve database integrity in a multi-user environment. Two kinds of concurrency control mechanisms are considered in this paper, namely optimistic and pessimistic ones. Optimistic concurrency control is based on the idea of conflicts and transaction restart while pessimistic concurrency control uses locking as the basic serialization mechanism. Analytic and simulation models of both mechanisms were developed in order to compare them as far as transaction response time is concerned. These results take into consideration parameters such as transaction arrival rate, number of database resources accessed per transaction, size of the database, system processing rate and degree of multiprogramming.

1. INTRODUCTION

A database may be considered as a finite collection of resources, subject to concurrent access by several users of a computer system. Users of a database management system interact with the database through *transactions* which are a set of read and write actions. A transaction, besides being the unit of user work, must also be considered the unit of consistency, in the sense that it takes the database from a consistent state into another consistent state [1]. In a multi-user environment, concurrent operations may destroy the consistency, or semantic integrity, of the database. Therefore, concurrency control mechanisms are necessary to preserve consistency in face of concurrent access.

Several concurrency control mechanisms for database systems have appeared in the literature during the past few years. Some of them use locking as the basic synchronization mechanism [1, 2]. The use of locking implies that every transaction must lock a resource in shared (exclusive) mode before reading (updating) it. A locking protocol is also needed to guarantee that a set of transactions which use locking can be run concurrently while preserving the database consistency. An example of such a protocol, called two-phase locking, appears in [1].

Two transactions are said to conflict when their concurrent execution may generate an inconsistent state for the database. Locking oriented mechanisms can be considered *pessimistic* since database resources are locked even though transactions may not conflict with other executing transactions. In other words, they are pessimistic because they may prevent transactions from executing concurrently, even though no synchronization anomaly occurs. These mechanisms will be referred to as Locking Oriented Concurrency Control (LOCC) ones.

Another type of mechanism is based on the idea that conflicts are infrequent situations and, therefore, appropriate actions to preserve database integrity should only be taken when they occur. This approach can be considered *optimistic* since database resources are not locked hoping that they will not be modified by other

transactions. Such mechanisms will be called Conflict Oriented Concurrency Control (COCC) ones. Examples of mechanisms along this line can be found in [3] for centralized databases and in [4, 5] for distributed ones.

The execution of a transaction under an optimistic CC mechanism may be divided into three phases: read phase, validation phase and possibly an update phase. During the read phase the transaction is processed and a list of the required updates to the database, called the write set, is built.

The list of resources read during this phase is called the read set. At the end of the read phase, the validation phase checks whether the updates, if applied to the database, preserve its semantic integrity. Besides the read and write sets, a special transaction numbering scheme is used to decide if the update phase may be allowed to proceed or if the transaction must be aborted. The validation phase is also needed for read-only transactions in order to guarantee that correct values will be retrieved from the database. Other forms of validations, such as timestamps, may also be used as it will be demonstrated in Appendix B of this paper.

There is still much to be done in the area of performance evaluation of concurrency control mechanisms. Some of the important published studies are [6-9] for centralized database and [10, 11] for distributed ones.

This paper presents a contribution to the area of concurrency control in centralized databases. A quantitative analysis of an optimistic timestamp-based concurrency control mechanism is presented. A two level modeling technique is employed making use of the theory of queueing networks and Markov chains. The validity of the model and of the assumptions made are verified by a simulation model.

The main performance measure used to evaluate optimistic and pessimistic concurrency control mechanisms in this paper is the average transaction response time. The results obtained are a function of database, transaction and computer system parameters,

such as size of the DB, transactions arrival rates, number of resources used by a transaction, CPU and IO service rates, multiprogramming degree and others.

We also present a comparative study of COCC with a pessimistic locking oriented concurrency control mechanism (LOCC). The results of the LOCC used in the comparison were obtained with the use of simulation.

This paper is organized as follows. Section 2 describes in detail the LOCC and COCC mechanisms considered here. Section 3 considers the basic analytic model, and the assumptions used in the analysis. The analytic model of the COCC mechanism is presented in section four. Next section presents some asymptotic results for the COCC case. The paper concludes with a discussion of the results obtained in our analysis. Appendix A contains the proofs of all analytic results in the paper and Appendix B contains an algorithmic description of a simple and efficient conflict oriented mechanism.

2. CONCURRENCY CONTROL MECHANISMS

This section describes the basic concurrency control mechanisms considered for evaluation in this paper. Rather than modeling specific concurrency control mechanisms the approach taken here was to adopt models general enough to represent the class of pessimistic and optimistic mechanisms.

Conflict oriented concurrency control

A transaction is divided into three phases: a *read phase*, a *computation phase* and a *test-and-update phase*.

During the read phase the transaction reads all necessary data (the read set). During the computation phase new values are generated for the database resources to be updated. In terms of available techniques for reliable update of databases, an intentions list for the transaction is recorded in stable storage in this phase (see [12, 13] for further details on this topic). When all computation is done, the test and update phase begins. During this phase, the system checks whether any resource of the read set was updated by another transaction. In the affirmative case, the transaction is rejected (its intentions list is discarded) and it must restart again from the read phase. Otherwise, the updates are committed (the intentions list is executed).

While the implementation of locking oriented concur-

rency control mechanisms is widespread and well understood, the same cannot be said regarding conflict oriented ones. Therefore, we include in Appendix B, a description of a straightforward implementation of a COCC mechanism. The interested reader will find out that all information needed in the test and update phase may be easily kept in main memory. Therefore, the time devoted to the test procedure is neglected in our analysis.

Figure 1 illustrates the computer system and the queue of transactions waiting for memory. Transactions which are rejected return to the computer system and those which succeed depart from it.

Locking oriented concurrency control

A lock is a serialization mechanism which protects the owner of the lock from modifications from other users. Therefore, while a lock on object *X* is active, no other user can access that object. Certain rules must be followed to lock and unlock objects in order to obtain serializable logs. Two phase locking[1] is an example of such rules.

The model considered here is a particular case of two phase locking in which all resources needed by a transaction are requested in advance and kept until the transaction terminates. Hence, deadlocks are prevented.

Our model of a LOCC mechanism is as follows. Upon arrival, a transaction is placed in a queue for memory if the number of executing transactions is equal to the degree of multiprogramming, otherwise the transaction will try to lock all resources it needs (Fig. 2). If any of the resources is not free the transaction enters a waiting queue for database resources. It should be emphasized that database resources, as considered here, are logical entities of the database and not physical ones. It is also assumed that lock tables are kept in main memory. Therefore, the time needed to check whether a transaction may be granted the requested resources is considered to be negligible. If the transaction succeeds in obtaining its resources, it starts executing immediately.

When a transaction departs, all the resources it held are released and the queue for DB resources is examined to remove from it those transactions which were blocked due to any of the resources just released. Also, the transaction in the front of the queue for memory is admitted into the computer system.

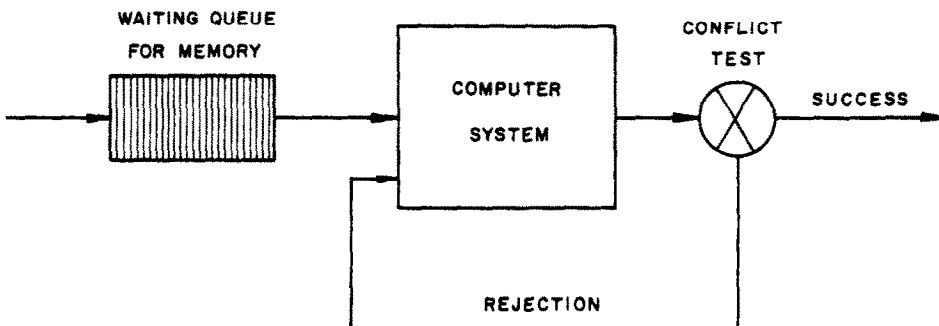


Fig. 1. COCC mechanism.

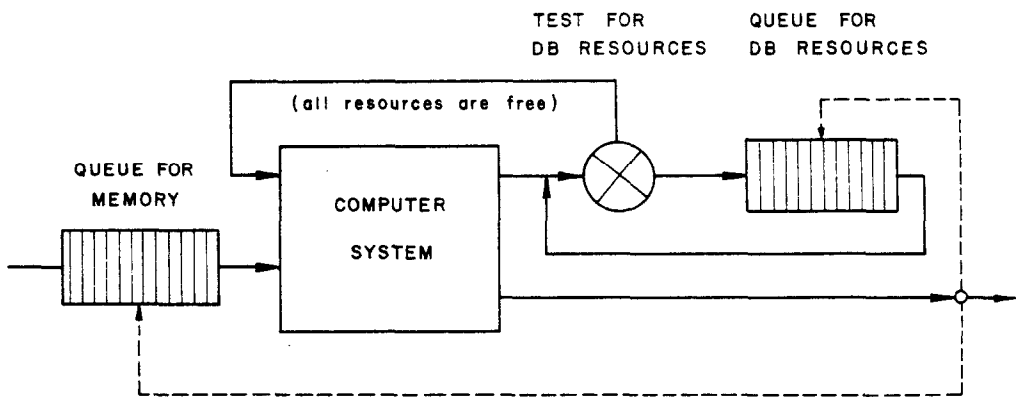


Fig. 2. LOCC mechanism.

3. MODEL AND PARAMETERS

This section presents the basic assumptions and parameters considered in the analysis of the concurrency control mechanisms discussed in the previous section. We will assume that the computer system is multiprogrammed. Therefore, each transaction will have to compete for memory, CPU and IO resources in addition to competing for DB resources. The set of transactions in execution (i.e. in memory) is called the *active set*. Let the maximum size of the active set, called the degree of multiprogramming, be equal to c . Therefore, an arriving transaction that finds c transactions in execution must wait (for memory) in a queue.

Computer system

The computer system is composed of one CPU and D identical and independent IO devices (disks). After receiving a certain amount of processing at the CPU, the transaction may require service from an IO device after which it may need more CPU or it may leave the computer system (Fig. 3).

Consider the following assumptions regarding the computer system:

CS1 After leaving the CPU the transaction selects one of the D IO devices with equal probability.

CS2 The service time of a transaction at an IO device is exponentially distributed with an average equal to $1/\mu_{IO}$.

CS3 The queueing discipline at each resource of the computer system (CPU or IO) is FCFS.

Database

In most studies of concurrency control mechanisms the DB is modeled as a collection of M resources called items [10, 11]. We will adopt the same model here.

Transactions

The lack of actual data regarding DB transactions have caused several discussions on the model to be adopted for a transaction. Several models have been considered [6-10]. Our assumptions regarding the transaction model are:

TM1 A transaction reads a set of DB items (read set) and updates the same set of items (write set). In other words, the read set is equal to the write set.

TM2 The write set (or read set) is composed of n items selected with equal probability among the M resources of the DB. This assumption may be easily relaxed as shown in the conclusions.

TM3 After receiving service at an IO device the trans-

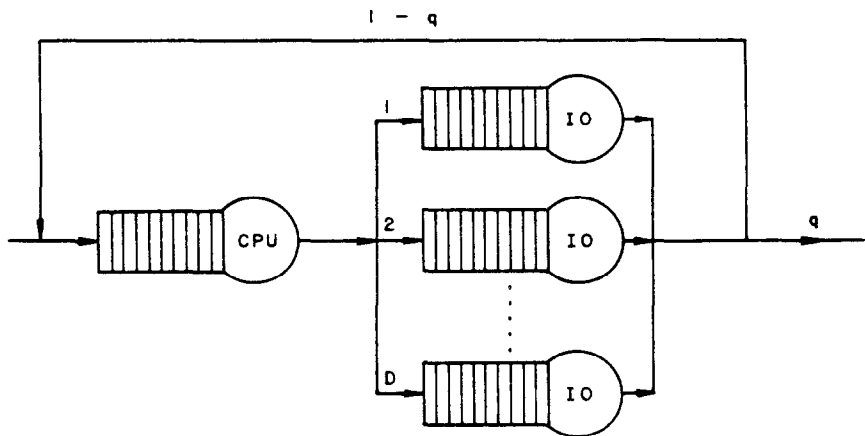


Fig. 3. Computer system.

- action returns to the CPU with probability $1 - q$ or leaves the system with probability q .
- TM4 The total CPU processing time of a transaction is exponentially distributed with an average equal to $1/\mu_{CPU}$. In the case of the COCC algorithm, this time should be maintained constant when the transaction is restarted. However, for the sake of simplifying our analytical model we assume that the total CPU processing time is resampled at each restart. Simulation results indicate that this is a very good assumption (see Conclusion).
- TM5 The IO access time is a function of the seek-time, rotational delay and data transfer delay. We assume that this time is exponentially distributed with average $1/\mu_{IO}$. Similar assumptions have been made in several studies of computer system performance[14].
- TM6 Transactions arrive according to a Poisson process with rate λ transactions per second.

Additional considerations

As mentioned already, we neglect here the overhead of the validation system for the COCC mechanism as well as the overhead for maintaining the locking system for the LOCC case. Ries[7] and Potier[9] studied the overhead of the locking system for the LOCC case.

Let us now make some considerations regarding transaction restarts in the COCC case.

- TR1 At each CPU-IO cycle the CPU service time is a constant fraction of the total CPU time.
- TR2 The number of IO accesses of a transaction is not necessarily the same each time a transaction is restarted. The reason behind this is that many database management systems such as IBM's System R and Medidata's MUMPS system use a buffer pool to store database records. Therefore, a

read or write request does not necessarily imply in an actual IO operation since the desired record may already be in the buffer pool (read case) or the buffer management system may choose not to write the buffer to disk at every single write request. The number of actual IO requests is translated in our model by the number of CPU-IO cycles performed by the transaction. This number is modeled as a geometrically distributed random variable with average $1/q$, and is resampled each time the transaction is restarted.

Parameter list

- M = number of data items of the database
 n = number of data items accessed per transaction
 λ = average transaction arrival rate
 D = number of IO devices in the computer system
 q = probability that a transaction leaves the computer system after being serviced at an IO device
 $1/\mu_{CPU}$ = average total CPU processing time of a transaction
 $1/\mu_{IO}$ = average IO service time per IO request
 c = degree of multiprogramming, i.e. the maximum number of transactions that may be sharing CPU and IO resources

Performance measures

The performance measure selected in our analysis of concurrency control mechanisms is the average response time.

Response time is defined in the COCC case as the time interval elapsed since the transaction arrival instant until the conflict test gives a successful result. At this point, we assume that an intentions list for the transaction is written in stable storage to be executed later on. See [12, 13] for further details on this matter.

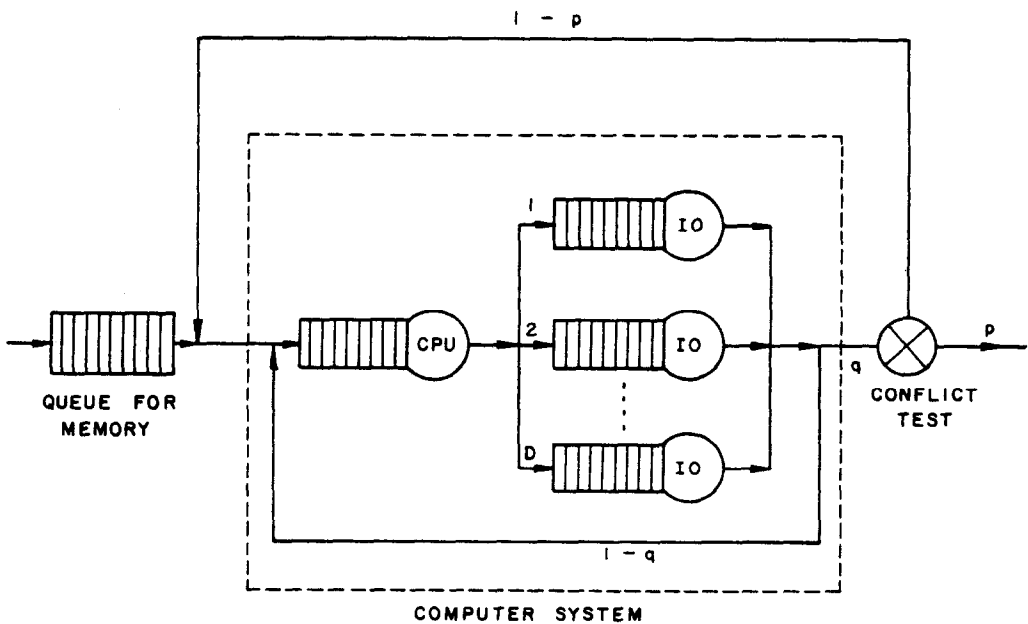


Fig. 4. Detailed COCC model.

For the LOCC case, the response time is measured since the arrival instant of a transaction until all resources locked by it are free.

4. ANALYTIC MODEL OF THE COCC MECHANISM

This section presents the modeling technique used to obtain the average response time of transactions under the parameters given in the previous section.

All proofs can be found in Appendix A.

Modeling technique

The technique used here consists in building a two level model which is used in an iterative procedure to obtain the desired results.

Let p be the probability that a transaction succeeds in its conflict test. Therefore, $1 - p$ is the probability that the transaction is restarted after leaving the computer system. Figure 4 depicts a detailed model of the COCC mechanism.

The value of the probability p is a function of the arrival rate λ and of the time a transaction spends in the computer system. On the other hand, this time is a function of the load in the system and therefore is also a function of p .

Our modeling approach can now be described as follows. Level 1 model considers that p is fixed and an approximation technique is used to obtain the average time a transaction spends in the computer system. Level 2 model substitutes the whole computer system by c parallel exponential servers with a known service rate. The resulting model is analyzed using a truncated Markov Chain model.

The results of the two levels are composed by an iterative procedure to be described later.

Level 1 analysis

We use here a technique similar to the one used by Avi-Itzhak [15] to model multiprogrammed systems. This technique consists in replacing the whole computer system along with the feedback loop due to restarts by a state dependent server with rate γ_k where k is the number of transactions in the system (including the queue for memory). Figure 5 shows the equivalent system to be analyzed.

Result 1. The state dependent service rate γ_k is given by the following expressions:

$$\gamma_k = \begin{cases} \frac{pqG(k-1)}{G(k)} & \text{for } k = 0, 1, \dots, c-1 \\ \frac{pqG(c-1)}{G(c)} & \text{for } k \geq c \end{cases} \quad (1)$$

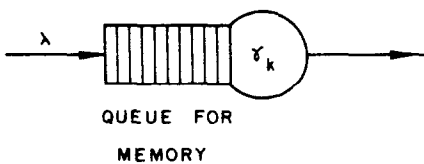


Fig. 5. COCC level 1 model.

where $G(k)$ is the normalization constant in the solution of the closed queueing network composed of the computer system plus the restart loop.

Result 2. The average execution time, E , of a transaction in the computer system is given by:

$$e = \frac{pP_0}{\lambda} \left\{ \frac{c\lambda^c}{\gamma_1 \dots \gamma_c \left(1 - \frac{\lambda}{\gamma_c}\right)} + \sum_{k=1}^{c-1} \frac{k\lambda^k}{\gamma_1 \dots \gamma_k} \right\} \quad (2)$$

where

$$P_0 = \left\{ 1 + \frac{\lambda^c}{\gamma_1 \dots \gamma_c \left(1 - \frac{\lambda}{\gamma_c}\right)} + \sum_{k=1}^{c-1} \frac{\lambda^k}{\gamma_1 \dots \gamma_k} \right\}^{-1} \quad (3)$$

Level 2 analysis

In this level, the whole computer system is substituted by c identical parallel exponential servers with average service rate equal to μ . Figure 6 shows this model. The whole system is now modeled as a Markov chain with states (i, j) where: i = number of transactions in the system (queue + computer system); and j = number of transactions in the computer system which can still complete successfully.

Let $P(i, j)$ be the probability that the system is in state (i, j) . For given values of c , λ and μ one can build the state transition diagram for the Markov chain defined above. The global balance equations plus the probability conservation equation form a system of linear equations whose solution yields the values of the probabilities $P(i, j)$. The number of equations in the system is infinite. However, in order to solve it numerically one must consider a finite number of equations. Let NMAX be the smallest integer such that $\Pr[i > \text{NMAX}] < \epsilon$ for a chosen tolerance $\epsilon \in \mathbb{R}$. In other words, NMAX can be considered as the maximum number of transactions in the system. Therefore if NMAX were known, one could build a diagram containing only states (i, j) where $i \leq \text{NMAX}$. Since NMAX is not known one can gradually increment the value of NMAX (therefore the size of the diagram is increased) until a new increment does not modify the solution within a given tolerance.

From the values of $P(i, j)$ one can easily obtain R and p .

State transition diagram (STD)

The STD can be obtained by applying the following rules:

Rule 1. For $1 \leq i \leq \min(c-1, \text{NMAX}-1)$ and $1 \leq j \leq i$ draw the state transitions shown in Fig. 7, where $q(j, k)/j$ is the probability that a departing transaction conflicts with exactly k transactions, out of a set of j running transactions that can still complete successfully. The expression for $q(j, k)$ is given by

$$q(j, k) = j \binom{j-1}{k} (1 - NC)^k NC^{j-1-k} \quad (4)$$

where NC is the probability that a given transaction does

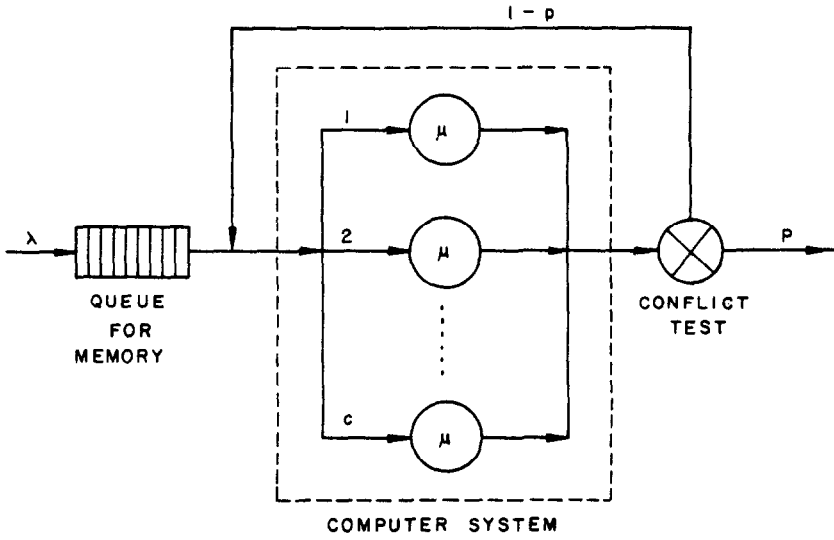


Fig. 6. COCC level 2 model.

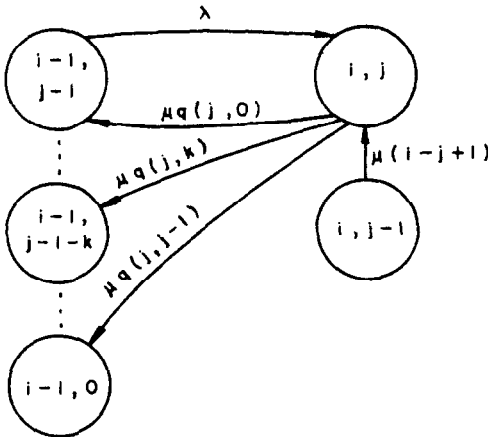


Fig. 7. Rules 1 and 2.

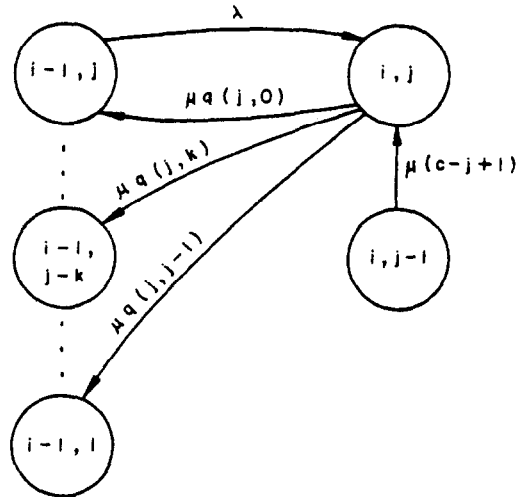


Fig. 9. Rule 4.

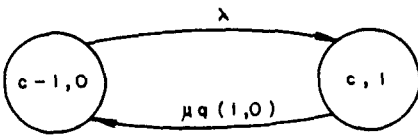


Fig. 8. Rule 3.

Rule 5. For $c < i \leq NMAX$ and $NMAX > c$ draw the transitions shown in Fig. 10.

Figure 11 shows an examples of a diagram for $c = 4$ and $NMAX = 5$.

not conflict with another transaction. This probability is given by the expression below

$$NC = \begin{cases} \binom{M-n}{n} / \binom{M}{n} & \text{if } n > M/2 \\ 0 & \text{if } n \leq M/2 \end{cases}$$

Rule 2. For $i = c$, $1 < j \leq c$ and $NMAX \geq c$ draw the state transitions shown in Fig. 7.

Rule 3. For $NMAX \geq c$ draw the state transitions shown in Fig. 8.

Rule 4. For $c < i \leq NMAX$, $1 < j \leq c$ and $NMAX > c$ draw the transitions shown in Fig. 9.

Flow balance equations

The set of global balance equations is given below:

(i) For $0 \leq i \leq \min(c-1, NMAX-1)$

$$(\lambda + i\mu)P(i, 0) - \mu \sum_{j=1}^{i+1} q(j, j-1)P(i+1, j) = 0 \quad (5)$$

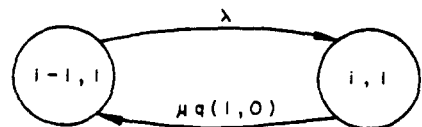


Fig. 10. Rule 5.

$$(\lambda + i\mu)P(i, j) - \lambda P(i-1, j-1) - (i-j+1)\mu P(i, j-1)$$

$$- \mu \sum_{k=j+1}^{i+1} q(k, k-j)P(i+1, k) = 0 \quad (6)$$

$$\text{for } 1 \leq j \leq i.$$

(ii) For $NMAX \leq c$ and $i = NMAX$

$$i\mu P(i, 1) - \lambda P(i-1, j-1) = 0 \quad (7)$$

$$(\lambda + i\mu)P(i, j) - \lambda P(i-1, j-1) - (i-j+1)\mu P(i, j-1) = 0 \quad (8)$$

$$\text{for } 1 < j \leq i.$$

(iii) For $NMAX > c$ and $i = c$

$$(\lambda + i\mu)P(i, 1) - \lambda P(i-1, 0) - \mu \sum_{j=1}^c q(j, j-1)P(i+1, j) = 0 \quad (9)$$

$$(\lambda + i\mu)P(i, j) - \lambda P(i-1, j-1) - (i-j+1)\mu P(i, j-1)$$

$$- \mu \sum_{k=j}^c q(k, k-j)P(i+1, k) = 0 \quad (10)$$

$$\text{for } 1 < j \leq c.$$

(iv) For $NMAX > c$ and $c < i < NMAX$

$$(\lambda + c\mu)P(i, 1) - \lambda P(i-1, 1) - \mu \sum_{j=1}^c q(j, j-1)P(i+1, j) = 0 \quad (11)$$

$$(\lambda + c\mu)P(i, j) - \lambda P(i-1, j) - (i-j+1)\mu P(i, j-1)$$

$$- \mu \sum_{k=j}^c q(k, k-j)P(i+1, k) = 0 \quad (12)$$

$$\text{for } 1 < j \leq c.$$

(v) For $NMAX > c$ and $i = NMAX$

$$i\mu P(i, 1) - \lambda P(i-1, 1) = 0 \quad (13)$$

$$i\mu P(i, j) - \lambda P(i-1, j) - (i-j+1)\mu P(i, j-1) = 0 \quad (14)$$

$$\text{for } 1 < j \leq c.$$

In order to solve the system, any one of the above equations must be substituted by the probability conservation equation, namely the sum of all probabilities $P(i, j)$ must be equal to one. The dimension, d , of the system is given by

$$d = \frac{\min(c, NMAX)[1 + \min(c, NMAX)]}{2} + \epsilon(NMAX - c)(NMAX + 1 - c)c \quad (15)$$

where

$$\epsilon(K) = \begin{cases} 1 & \text{if } K \geq 0 \\ 0 & \text{otherwise.} \end{cases}$$

The probability, p , that a transaction completes successfully is given by

$$p = \frac{\sum_{i=1}^{NMAX} \sum_{j=1}^i jP(i, j)}{\sum_{i=1}^{NMAX-1} i \sum_{j=0}^i P(i, j) + NMAX \sum_{j=1}^{NMAX} P(NMAX, j)} \quad (16)$$

if $NMAX \leq c$

and by

$$p = \frac{\sum_{i=1}^{c-1} \sum_{j=1}^i jP(i, j) + \sum_{i=c}^{NMAX} \sum_{j=1}^c jP(i, j)}{\sum_{i=1}^{c-1} i \sum_{j=0}^i P(i, j) + \sum_{i=c}^{NMAX} i \sum_{j=1}^c P(i, j)} \quad (17)$$

if $NMAX > c$.

The average response time, R , can be easily obtained using

$$R = \frac{1}{\lambda} \left[\sum_{i=1}^{NMAX-1} i \sum_{j=0}^i P(i, j) + NMAX \sum_{j=1}^{NMAX} P(NMAX, j) \right] \quad (18)$$

if $NMAX \leq c$

and

$$R = \frac{1}{\lambda} \left[\sum_{i=1}^{c-1} i \sum_{j=0}^i P(i, j) + \sum_{i=c}^{NMAX} i \sum_{j=1}^c P(i, j) \right] \quad (19)$$

if $NMAX > c$.

Composition of the two levels

For a given value of p , the results of level 1 give us the average execution time, E , of a transaction within the computer system. On the other hand, for a given value of the rate of execution of a transaction within the computer system, level 2 model allows us to obtain the value of the probability p and of the response time R .

The composition of the two levels is done by the following procedure:

(i) Select an initial value $p^{(0)}$ for the probability p . Set $i = 0$. A recommended value for $p^{(0)}$ is 0.9999.

(ii) Compute E using $p = p^{(0)}$ in Results 1 and 2 of the level 1 model.

(iii) Set $i = i + 1$. Find a new value $p^{(i)}$ for the probability p using the level 2 model where $1/\mu$ is made equal to E .

(iv) If the relative error $|(p^{(i)} - p^{(i-1)})/p^{(i)}|$ is greater than a given tolerance δ then return to step ii.

(v) Compute the average response time using expressions (18) and (19) of the level 2 model.

5. ASYMPTOTIC RESULTS

We present here some asymptotic results which can be used to have a rough idea of the response time curve

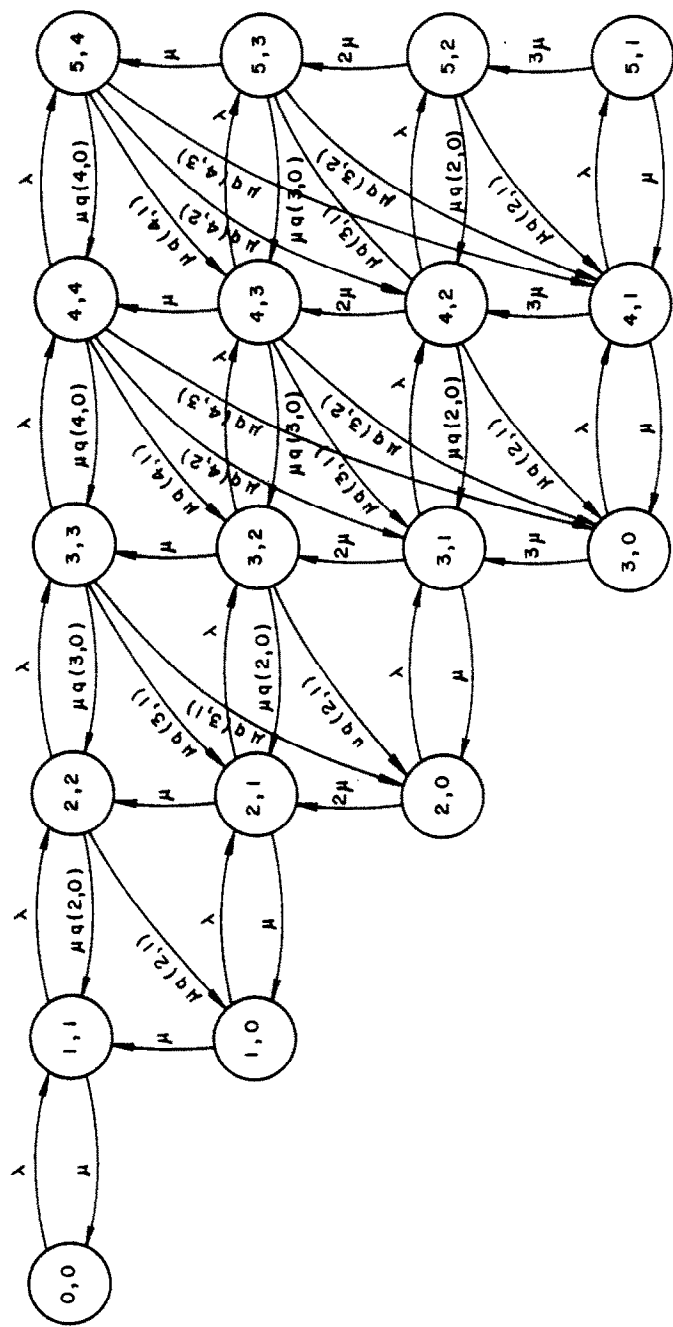


Fig. 11. Example of a complete state transition diagram ($c = 4$ and $NMAX = 5$).

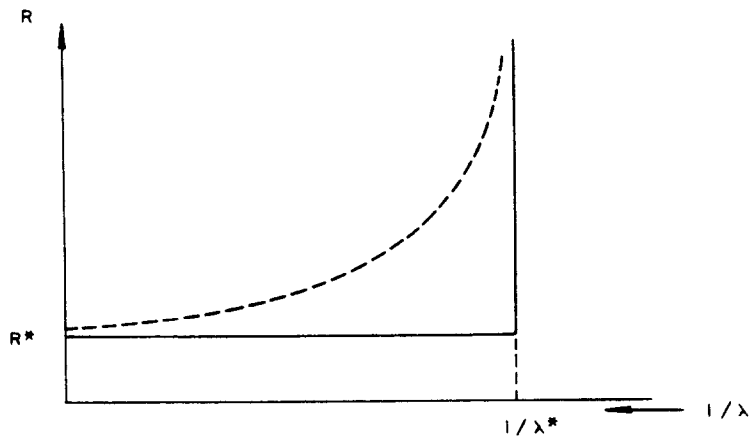


Fig. 12. Asymptotes of COCC.

(Fig. 12). Two extreme cases are considered: lightly loaded and heavily loaded system.

Almost empty system

Here we consider the case where there is virtually no interference between the transactions. Hence, the average response time is trivially given by

$$R^* = \frac{1}{\mu_{CPU}} + \frac{1}{q\mu_{IO}}. \tag{20}$$

Heavy loaded system

In this case we consider that there is always a backlog of transactions ready to enter the active set. Therefore, there are always *c* transactions in the computer system.

Under these circumstances we can calculate the asymptotic value of the probability, *p_c*, of successful completion of a transaction for a heavily loaded system. We will also be able to find the maximum value of λ, λ*, for which the system is stable.

Result 3. (COCC—Heavy Load). The value of *p_c* is given by

$$p_c = \frac{\sqrt{(1+4(c-1)(1-NC))}-1}{2(c-1)(1-NC)} \text{ for } c > 1. \tag{21}$$

Result 4. (COCC—Heavy Load). The value of λ* is given by

$$\frac{1}{\lambda^*} = \frac{G(c)}{p_c q G(c-1)}. \tag{22}$$

Table 1 shows several values of *R** and 1/λ* for different values of the system parameters.

CONCLUSIONS

A two-level analytic model was developed to study the performance of a conflict oriented concurrency control mechanism. The average response time of transactions, *R*, was obtained as a function of different parameters.

Table 1. COCC asymptotes

c	EXAMPLE 1		EXAMPLE 2	
	PARAM. and R*	1/λ*	PARAM. and R*	1/λ*
2	M=200	10.47	M=100	7.16
3		9.13		6.27
4	n=5	8.58	n=5	5.88
5	D=3	8.34	D=4	5.70
6	q=0.2	8.25	q=0.25	5.61
7	1/μ _{CPU} =2.5	8.24	1/μ _{CPU} =2.0	5.59
8	1/μ _{IO} =2.5	8.28	1/μ _{IO} =2.0	5.59
9	R*=15.0	8.35	R*=10.0	5.62
10		8.44		5.67
12		8.66		5.78
14		8.89		5.92
16		9.14		6.06
18		9.39		6.21

It can be observed that R is sensitive to the degree of conflicts between transactions and to various parameters of the computer system. In particular R is sensitive to variations of M , n , q , D and $1/\mu_{IO}$. For instance, Fig. 13 displays R as a function of $1/\lambda$ for three values of the size of the database ($M = 50$, $M = 100$ and $M = 200$). The three asymptotes are also shown. In this picture one can see the effect of the degree of conflicts in the average response time. The probability of conflict be-

tween two transaction $(1 - NC)$ for $M = 50$ and $n = 5$ is 0.423 while for $M = 200$ and $n = 5$ is equal to 0.1201. As it can be seen from Fig. 13, R is 43% times greater for $M = 50$ than for $M = 200$ for $1/\lambda = 20$.

A similar effect can be observed in Fig. 14 which shows the variation of R with $1/\lambda$ for several values of n .

Figure 15 illustrates the influence of the degree of multiprogramming, c , in the average response time. An interesting effect can be observed in this figure, as one

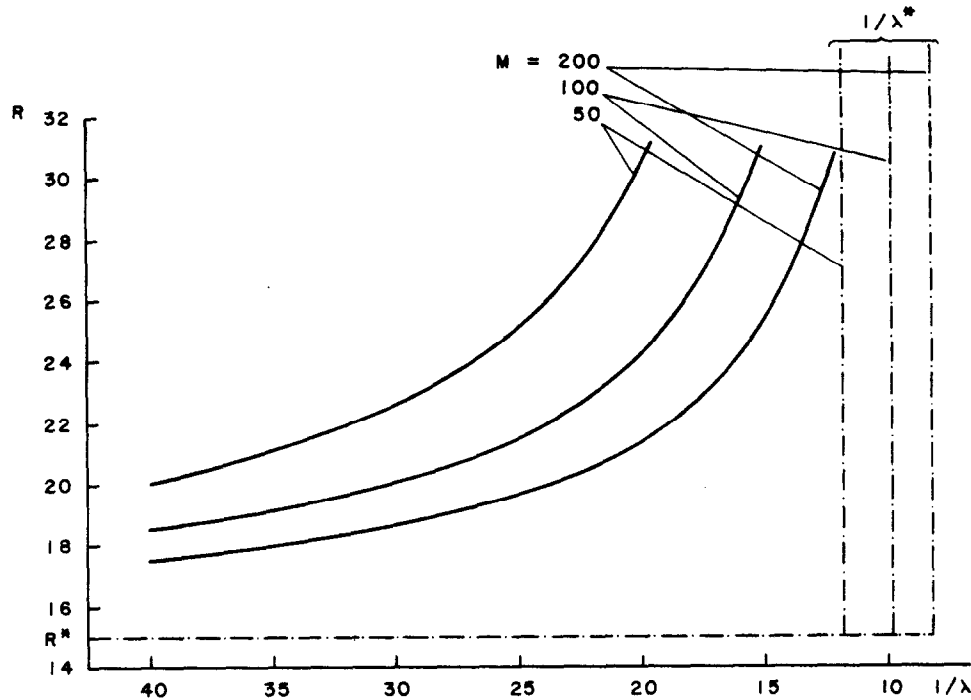


Fig. 13. COCC: average response time vs $1/\lambda$ for different values of M . ($n = 5$; $D = 3$; $c = 6$; $q = 0.2$; $1/\mu_{CPU} = 2.5$; $1/\mu_{IO} = 2.5$).

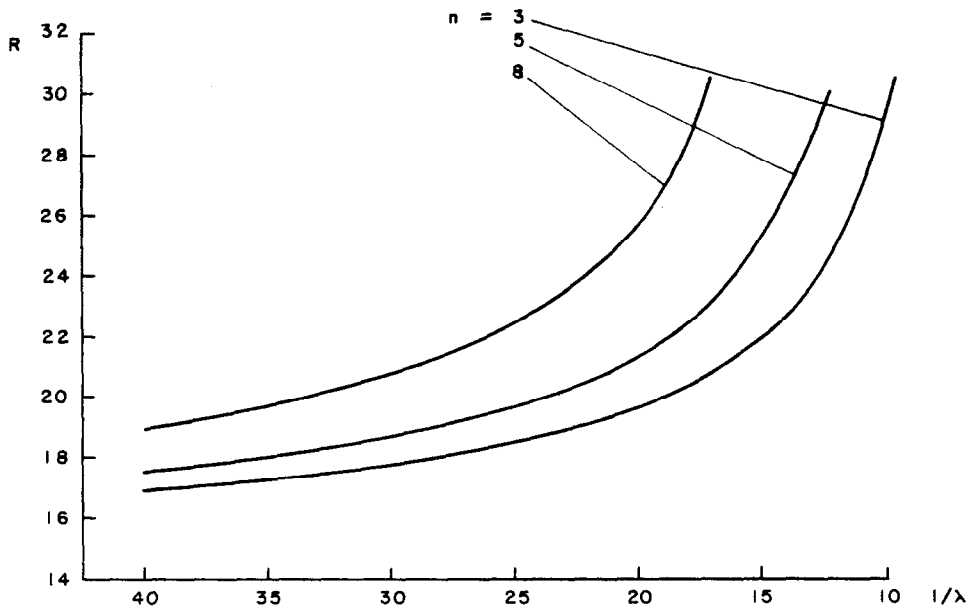


Fig. 14. COCC: average response time vs $1/\lambda$ for different values of n . ($M = 200$; $D = 3$; $c = 6$; $q = 0.2$; $1/\mu_{CPU} = 2.5$; $1/\mu_{IO} = 2.5$).

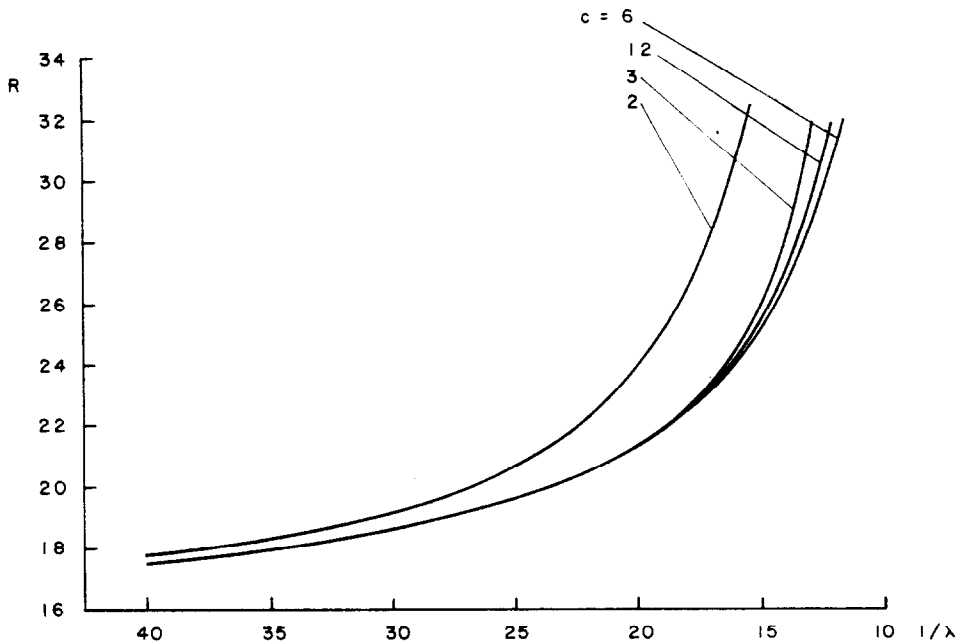


Fig. 15. COCC: average response time vs $1/\lambda$ for different values of c . ($M=200$; $n=5$; $D=3$; $q=0.2$; $1/\mu_{CPU}=2.5$; $1/\mu_{IO}=2.5$).

increases the degree of multiprogramming from 2 to 12.

It can be seen from the figure that the response time for $c=3$ is better than for $c=2$ but the response time for $c=12$ is worse than for $c=6$ for instance. This behaviour can be better observed in Fig. 16 which shows the variation of R with c for several values of $1/\lambda$. As c increases, R decreases, then it increases again until it reaches a constant value. This behaviour is basically due to the variation of the size of the external queue (queue for memory) with c , as explained intuitively below.

There are three factors which influence the response time of a transaction. First, we have the waiting time in the external queue. An increase in c implies in a decrease in this waiting time and in the response time. The second factor is the contention for CPU and IO resources in the computer system which increases with c implying in an increase in the response time. Thirdly as we have more transactions in execution, the probability of successful completion of a transaction decreases and the response time increases. Now, these three factors

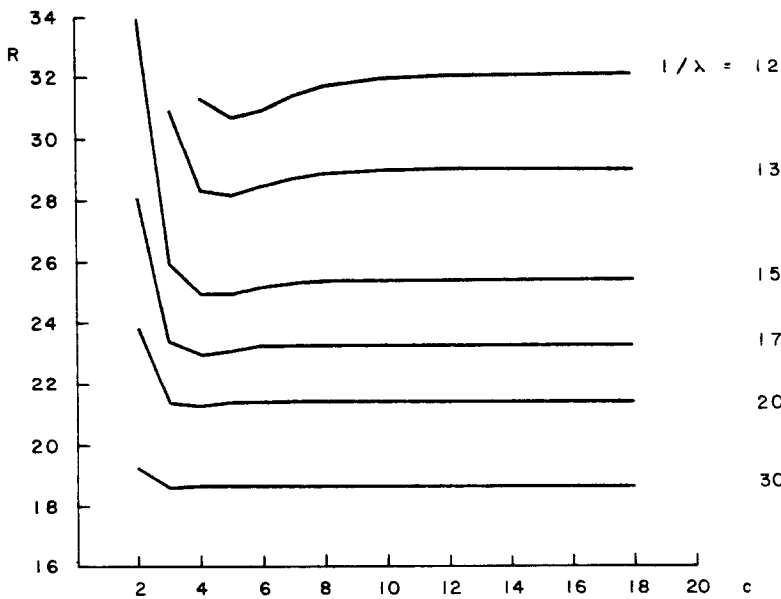


Fig. 16. COCC: average response time as a function of c . ($M=200$; $n=5$; $D=3$; $q=0.2$; $1/\mu_{CPU}=2.5$; $1/\mu_{IO}=2.5$).

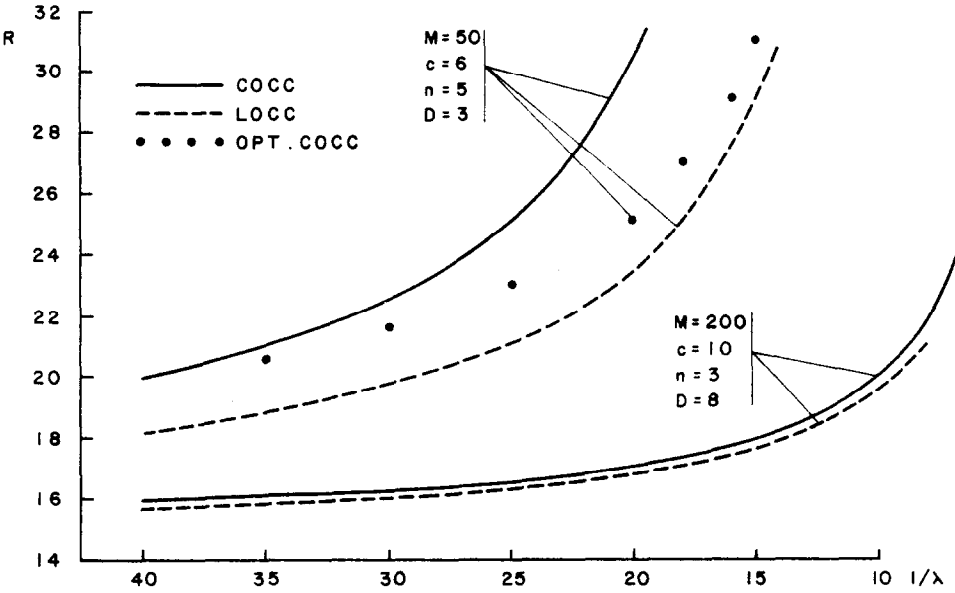


Fig. 17. COCC vs LOCC ($q = 0.2$; $1/\mu_{CPU} = 2.5$; $1/\mu_{IO} = 2.5$).

cannot be analyzed separately but in conjunction. For small values of c (e.g. $c = 2$) the time spent by a transaction in the external queue may be a significant portion of its response time. As c increases the waiting time in this queue decreases and the response time will decrease as long as this factor dominates the two other factors described above. When this trend is reversed, the response time will start to rise until a further increase in the value of c is not translated by a corresponding increase in the number of executing transactions. At this point, R will become constant with c for a given value of $1/\lambda$.

The effect described above can also be observed for the values of the asymptotes for COCC, as shown in Table 1.

Let us now comment on the comparison between conflict oriented concurrency control and locking oriented concurrency control mechanisms. A simulation model was built to obtain results for the LOCC model described in Sections 2 and 3. Figure 17 shows the average response time for both COCC and LOCC

mechanisms for two different sets of parameters. The dashed curves stand for the LOCC model. In both cases, LOCC has a better performance than COCC. This is true in all situations we analyzed. However, there are situations in which COCC can be almost as good as LOCC as far as response time is concerned (see the two lowermost curves of Fig. 17). This occurs in situations where the degree of conflict between transactions is low and when the computer system is such that the extra load due to transaction restarts can be easily absorbed by the available CPU and IO resources. This is the situation in Fig. 17 for the case in which $M = 200$ and $n = 3$ and $D = 8$, i.e. the probability of conflict between two transactions is $1 - NC = 0.0445$ and there are eight IO devices when $c = 10$.

The analytic model for the COCC mechanism was validated by a simulation model. Let us remind the reader again that in the simulation model, unlike the analytic model, the total CPU time of a transaction is held constant throughout all of its restarts. Table 2

Table 2. Comparison between analytic and simulation results for the COCC

n=5; c=6; D=3; q=0.2; 1/μCPU=2.5; 1/μIO=2.5									
M	1/λ	ANALYT.	SIMUL.	DIF.	M	1/λ	ANALYT.	SIMUL.	DIF.
200	40	17.55	17.31	-1.39	50	40	20.05	19.64	-2.09
	30	18.64	18.42	-1.19		35	21.06	20.70	-1.74
	25	19.63	19.40	-1.19		30	22.57	22.51	-0.27
	20	21.37	21.50	0.60		25	25.15	25.46	1.22
	18	22.50	22.69	0.84		23	26.76	27.46	2.55
	17	23.23	23.34	0.47		22	27.79	28.91	3.87
	16	24.12	24.20	0.33		21	29.00	30.56	5.10
	15	25.21	25.60	1.52		20	30.48	32.12	5.11
	14	26.62	26.77	0.59		19	32.31	34.52	6.40
	13	28.47	29.22	2.57		18	34.65	38.70	10.47
200	12	31.03	33.60	7.65					
	11	34.95	39.69	11.94					

displays analytic and simulation results and shows that the analytic model remarkably accurate. Percentual differences remain below the 10% level in most of the range of interest. It should also be noted that good results are obtained for both low and high probability of conflicts between transactions ($M = 200$, $n = 5$ and $M = 50$, $n = 5$), as shown in Table 2.

Another consideration concerns the size of the read and write sets of the transactions. In our model we assumed that the read and write sets are the same and that their size is constant. However, this is not a limitation of our model. Other distributions of the size of the read (or write) set can be easily accommodated, by properly computing the value of NC (the probability that two transactions do not conflict). For instance, if the size of the read (write) set is geometrically distributed with the following distribution

$$P[\tilde{n} = i] = \frac{1}{n} \left(1 - \frac{1}{n}\right)^{i-1} \quad \text{for } i = 0, 1, 2, \dots$$

where $\tilde{n}$ is the random variable that indicates the size of the read (write) set, the probability that two transactions do not conflict, NC , is given by the expression below

$$NC = \frac{1}{n^2} \sum_{i=1}^{M-1} \sum_{j=1}^{M-i} \frac{\binom{M-i}{j}}{\binom{M}{j}} \left(1 - \frac{1}{n}\right)^{i+j-2}. \quad (23)$$

Finally, we would like to add that further refinements and considerations about the COCC model presented here can be made. For instance, a possible optimization of the COCC mechanism would be to stop immediately the execution of the transactions which conflict with a transaction which completes successfully. In this way their execution may be restarted earlier and CPU and IO resources may be better utilized. The black dots in Fig. 17 display the results for the average response time for this optimized version of the COCC mechanism. These results were obtained with the use of a simulation model.

Acknowledgements—The authors would like to express their gratitude to Gaston H. Gonnet, Marco A. Casanova, Lazlo Belady and Richard R. Muntz who read a first version of this paper and made several useful comments. Gratitude is also due to the referees who contributed a lot to improve the quality of this paper.

REFERENCES

- [1] K. P. Eswaran, J. N. Gray, R. A. Lorie and I. L. Traiger: The notions of consistency and predicate locks in a database system. *CACM* Nov. (1976).
- [2] J. N. Gray: *Notes on Database Operating Systems: An Advanced Course*, pp. 394-481. Springer-Verlag Berlin (1978).
- [3] H. T. Kung and J. Robinson: On optimistic methods for concurrency control. *Proc. VLDB 1979*, Rio de Janeiro (Oct. 1979).
- [4] D. Z. Badal: Correctness of concurrency control and implications in distributed databases. *Proc. 1979 IEEE COM-PSAC*, Chicago (Nov. 1979).
- [5] R. H. Thomas: A majority consensus approach to concurrency control. *ACM TODS*, 4(2), 180-209 (1979).
- [6] D. R. Ries and M. R. Stonebraker: Effects of locking granu-

- larity in a database management system. *ACM TODS* 2(3), 233-246 (1977).
- [7] D. R. Ries and M. R. Stonebraker: Locking granularity revisited. *ACM TODS* 4(2), 210-227 (1979).
- [8] K. B. Irani and H. L. Lin: Queueing network model for concurrent transaction processing in a database system. *Proc. 1979 ACM SIGMOD Int. Conf. Management of Data*, Boston, Mass. (June 1979).
- [9] D. Potier and Ph. Leblanc: Analysis of locking policies in database management systems. *CACM* (Oct. 1980).
- [10] H. G. Molina: Performance of update algorithms for replicated data in a distributed database. Ph.D. Dissertation, Computer Science Department, Stanford University (June 1979).
- [11] J. E. R. Dantas: Performance analysis of distributed database systems. Ph.D. Dissertation, Computer Science Department, University of California, Los Angeles (1980).
- [12] B. Lampson and H. Sturgis: Crash recovery in a distributed data storage system. *Technical Report XEROX*, Palo Alto Research Center, California (1976).
- [13] D. A. Menascé and O. E. Landes: On the design of a reliable storage component for distributed database management systems. *Proc. 6th VLDB*, Montreal, Canada (Oct. 1980).
- [14] L. Lipsky and J. D. Church: Application of a queueing network model for a computer system. *ACM Computing Surveys* 4(3), 205-222 (1977).
- [15] B. Avi-Itzhak and D. P. Heyman: Approximate queueing models for multiprogramming computer systems. *Ops. Res.* 21(6), 1212-1230 (1973).
- [16] L. Kleinrock, *Queueing Systems: Computer Applications*, Vol. 2. Wiley Interscience, New York (1976).
- [17] J. P. Buzen, Computational algorithms for closed queueing networks with exponential servers. *CACM* 16, 527-531 (1973).

APPENDIX A

Proof of Result 1

Let us first model the computer system as a closed queueing network of the central server type (see Fig. A1) with K' transactions.

Using Gordon and Newell's notation[16] we have that the solution to the closed network defined above is given by

$$p(k_1, k_2, \dots, k_{D+1}) = \frac{1}{G(K')} \prod_{i=1}^{D+1} x_i^{k_i} \quad (A1)$$

where $p(k_1, \dots, k_{D+1})$ is the equilibrium probability of finding k_1 transactions at server 1 (CPU), $\dots$, k_i transactions at server i , $\dots$, k_{D+1} at device $D+1$. $G(K')$ is a normalization constant given by the expression below.

$$G(K') = \sum_{\mathbf{K}} \prod_{i=1}^{D+1} x_i^{k_i} \quad (A2)$$

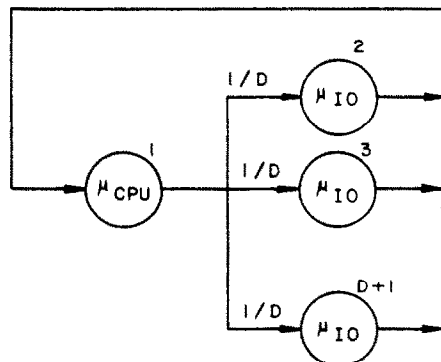


Fig. A1. Computer system as a central server model.

where A is the set of all tuples $(k_1, \dots, k_{D+1})$ such that $\sum_{i=1}^{D+1} k_i = K'$. This normalization constant can be efficiently obtained using Buzen's convolution algorithm[17].

The x_i 's which appear in (A1) must satisfy the system of equations below

$$\mu_1 x_1 = \sum_{i=2}^{D+1} \mu_i x_i \quad (A3)$$

$$\mu_i x_i = \frac{\mu_1 x_1}{D} \quad \text{for } i = 2, \dots, D+1$$

where $\mu_1 = \mu_{\text{CPU}}^{\Delta} \triangleq \mu_{\text{CPU}}/q$ and $\mu_i = \mu_{\text{IO}}$ for $i = 2, \dots, D+1$.

Setting $\mu_1 x_1 = 1$ we get

$$\begin{aligned} x_1 &= 1/\mu_{\text{CPU}} \text{ and} \\ x_i &= 1/D\mu_{\text{IO}} \text{ for } i = 2, \dots, D+1. \end{aligned} \quad (A4)$$

The CPU utilization, ρ_{CPU} , can be easily obtained in terms of the normalization constant[16], as indicated in the following expression.

$$\rho_{\text{CPU}} = \frac{1}{\mu_{\text{CPU}}} \frac{G(K'-1)}{G(K')}. \quad (A5)$$

The average rate at which transactions arrive at the CPU, λ_{CPU} , is equal to the rate at which they complete CPU-IO cycles, and is given by

$$\lambda_{\text{CPU}} = \rho_{\text{CPU}} \mu_{\text{CPU}} = \frac{G(K'-1)}{G(K')}. \quad (A6)$$

If we let $T(K')$ be the average time spent by a transaction to complete a CPU-IO cycle, it follows from Little's Result that

$$T(K') = \frac{K'}{\lambda_{\text{CPU}}} = \frac{K' G(K')}{G(K'-1)}. \quad (A7)$$

Each time a transaction is executed, it performs $1/q$ CPU-IO cycles on the average. Since the average number of restarts is $1/p$ we have that the average time, $A(K')$, spent in the computer system by a transaction when there are K' transactions in the computer system is given by

$$A(K') = \frac{1}{p} \cdot \frac{1}{q} \cdot T(K') = \frac{K' G(K')}{pq G(K'-1)}. \quad (A8)$$

Little's Result along with expression (A8) can now be used to obtain $\gamma_{K'}$, the rate at which successful transactions leave the computer system.

$$\gamma_{K'} = \frac{K'}{A(K')} = \frac{pq G(K'-1)}{G(K')} \quad \text{for } K' = 0, 1, \dots, c. \quad (A9)$$

Finally, if we let k be the total number of transactions in the system (queue + computer system) we get the desired result, namely that,

$$\gamma_k = \begin{cases} \frac{pq G(k-1)}{G(k)} & \text{for } k = 0, \dots, c-1 \\ \frac{pq G(c-1)}{G(c)} & \text{for } k \geq c. \end{cases} \quad (A10)$$

Proof of Result 2

Let P_k be the equilibrium probability of finding k transactions in the system (queue + computer system). Modelling the system of Fig. 5 as a birth and death process with birth rate equal to λ and death rate equal to γ_k we get the following values for the probabilities P_k .

$$P_k = \begin{cases} \frac{P_0 \lambda^k}{\gamma_1 \dots \gamma_k} & k = 1, 2, \dots, c \\ P_c \left(\frac{\lambda}{\gamma} \right)^{k-c} & k > c \end{cases} \quad (A11)$$

$$P_0 = \left[1 + \frac{\lambda^c}{\gamma_1 \dots \gamma_c \left(1 - \frac{\lambda}{\gamma_c} \right)} + \sum_{k=1}^{c-1} \frac{\lambda^k}{\gamma_1 \dots \gamma_k} \right]^{-1}. \quad (A12)$$

The average number of transactions in the computer system, $\bar{N}_{cs}$, is given by

$$\begin{aligned} \bar{N}_{cs} &= \sum_{k=1}^{c-1} k P_k + c \sum_{k=c}^{\infty} P_k \\ &= P_0 \left[\frac{c \lambda^c}{\gamma_1 \dots \gamma_c \left(1 - \frac{\lambda}{\gamma_c} \right)} + \sum_{k=1}^{c-1} \frac{k \lambda^k}{\gamma_1 \dots \gamma_k} \right]. \end{aligned} \quad (A13)$$

The average total time a transaction spends in the system is given by $\bar{N}_{cs}/\lambda$ and the average execution time, E , of a transaction (not counting the restarts) is given by $p \bar{N}_{cs}/\lambda$ for a given value of p . Hence,

$$E = \frac{p P_0}{\lambda} \left[\frac{c \lambda^c}{\gamma_1 \dots \gamma_c \left(1 - \frac{\lambda}{\gamma_c} \right)} + \sum_{k=1}^{c-1} \frac{k \lambda^k}{\gamma_1 \dots \gamma_k} \right] \quad (A14)$$

Proof of Result 3

Let us consider that there are c transactions in execution, i.e. as soon as a transaction completes successfully it is immediately replaced by another. The probability of success p_c can be obtained as follows.

Let T be the execution time of one of the c transactions (the flagged one). We consider now an approximate model for the COCC model which views the computer system as c parallel exponential servers with rate μp_c each. Therefore, the process of successful transaction completions is a Poisson process and the rate at which the other $(c-1)$ transactions complete successfully is given by

$$\mu(c-1)p_c \quad (A15)$$

If we now consider those transactions which complete successfully and conflict with our tagged transaction we get a Poisson process with rate

$$\mu(c-1)(1-NC)p_c \quad (A16)$$

Therefore, $P(T) = P[\text{none of the remaining transactions conflict with our tagged transaction during its execution time}]$

$$= e^{-\mu(c-1)p_c(1-NC)T}. \quad (A17)$$

Unconditioning in T we get

$$p_c = \int_{T=0}^{\infty} P(T) \mu e^{-\mu T} dT = \frac{1}{(c-1)(1-NC)p_c + 1}. \quad (A18)$$

Solving for p_c we get

$$p_c = \frac{\sqrt{(1+4(c-1)(1-NC))}-1}{2(c-1)(1-NC)} \quad \text{for } c > 1. \quad (A19)$$

Proof of Result 4

The proof to this result is trivial. In order for the Markov chain defined in Section 4 (Level 1 analysis) to reach equilibrium there must be a state k_0 such that $\lambda/\gamma_k < 1$ for $k \geq k_0$. Therefore, from the values for γ_k as given in eqn (1), it follows that $\lambda < \gamma_c$.

Hence,

$$\lambda^* = \frac{p_c q G(c-1)}{G(c)}. \quad (A21)$$

APPENDIX B

A simple implementation of the COCC mechanism is given here. This implementation demonstrates that all information necessary to test whether a transaction must be rejected or not may be kept in main memory in two tables defined below:

Resource state table (RST)

An entry in RST is given by the triple (R, n, t) where R = resource identification; n = number of transactions in execution which read the value of resource R ; and t = timestamp of resource R , i.e. the time at which the last version of resource R was generated.

Transaction status table (TST)

An entry in TST is given by the pair (T, S) where T = transaction identification; and $S = \{(R_{i_1}, t_{i_1}), \dots, (R_{i_n}, t_{i_n})\}$ where t_{i_k} for $k \in \{1, \dots, n\}$ is the timestamp of resource R_{i_k} in RST right after the resource was read by transaction T .

The implementation is described below, in PASCAL like, by two procedures READPH and TESTPH which are executed during the read and test-and-update phases respectively. In what follows, CLOCK is a function which returns the value of the current time.

procedure READPH;

{consider a transaction TX which reads resources $R_{i_1}, \dots, R_{i_n}$ }

Begin

create entry in TST for transaction TX;

for each $R \in \{R_{i_1}, \dots, R_{i_n}\}$ do

if R is in RST

then begin

RST(R). n := RST(R). n + 1;

TST(TX). S := TST(TX). S $\cup \{(R, \text{RST}(R).t)\}$

end

else begin

create entry for resource R in RST;

RST(R). n := 1;

RST(R). t := CLOCK;

TST(TX). S := TST(TX). S $\cup \{(R, \text{CLOCK})\}$

end

end {of procedure READPH}

If a resource R has not been read by any executing transaction, the value of the CLOCK is assigned as the time-stamp for resource R . Otherwise, the timestamp of R in table RST is the one considered for inclusion in TST. Procedure TESTPH is given below.

Procedure TESTPH;

{let TX be the transaction in the test-and-update phase that executes this procedure}

begin

update := yes;

for every $(R, t) \in \text{TST}(\text{TX}).S$ do

begin

if $t < \text{RST}(R).t$

then update := no;

RST(R). n := RST(R). n - 1;

if RST(R). n = 0

then delete entry of R in RST;

end

if update = yes

then for every $(R, t) \in \text{TST}(\text{TX}).S$ do

RST(R). t := CLOCK;

end{of TEST PH} delete entry of TX in TST

Some comments are in order. The database is updated right after the execution of TESTPH if the value of update is yes. Exclusive access to the RST and TST table should be guaranteed to the transaction which executes the procedure TESTPH. This may be achieved through the use of synchronization primitives available in most operating systems. Recall that tables RST and TST are in main memory. Another interesting feature of the protocol described above is that timestamps need not be stored in the database since they only exist for resources which are being used by any transaction in execution.